

LABORATORY RECORD

PYTHON AUTOMATION

Milestone 2 — Unit Test Frameworks & Page Object Model

Submitted by

Pratyay Chowdhury

Enrollment No.: 12023052020034

B.Tech, Computer Science & Engineering (AI)

INSTITUTE OF ENGINEERING AND MANAGEMENT

Department of Computer Science & Engineering (AI)

Kolkata, West Bengal

Index

Sl. No.	Experiment / Section Title	Page
1	Introduction to Unit Test Frameworks & Page Object Model	3
2	Experiment 7: Login Automation Script	3
3	Experiment 8: Data-Driven Testing (DDT) using External CSV	5
4	Experiment 9: PyTest Integration with HTML Reporting	8
5	Overall Summary	10

Introduction to Unit Test Frameworks & Page Object Model

Unit testing and test-framework concepts provide structure around Selenium automation by making individual checks easier to execute, verify and maintain. PyTest provides test discovery and execution facilities, while external test data can be used to repeat a test flow with multiple input combinations. The Page Object Model is a design approach in which page-specific interactions are kept separate from the test flow.

This record covers Milestone 2 through three practical assignments: Login Automation Script, Data-Driven Testing using an external CSV file, and PyTest Integration with HTML Reporting. All three assignments use the SauceDemo practice application, and the execution evidence is reproduced from the original terminal and browser sessions.

Common Environment Setup

The following setup applies to all experiments unless stated otherwise:

Python with Selenium WebDriver.

Google Chrome browser.

SauceDemo practice application: <https://www.saucedemo.com>

CSV file for external test data in Assignment 8.

PyTest and pytest-html for Assignment 9.

Experiment 7: Login Automation Script

1. Problem Statement

Automate the login flow on the SauceDemo application by launching the browser, maximizing the window, navigating to the application, submitting valid credentials and validating that the resulting URL matches the expected post-login inventory page.

2. Objective

Launch the Chrome browser and open the SauceDemo application.

Enter valid login credentials and submit the login form.

Validate the resulting inventory-page URL after login.

3. Tools, Software and Concepts Used

Python and Selenium WebDriver.

Chrome browser.

SauceDemo practice application.

Element locators using Selenium By.ID.

4. Implementation Steps

Create a Chrome WebDriver instance and maximize the browser window.

Navigate to the SauceDemo login page.

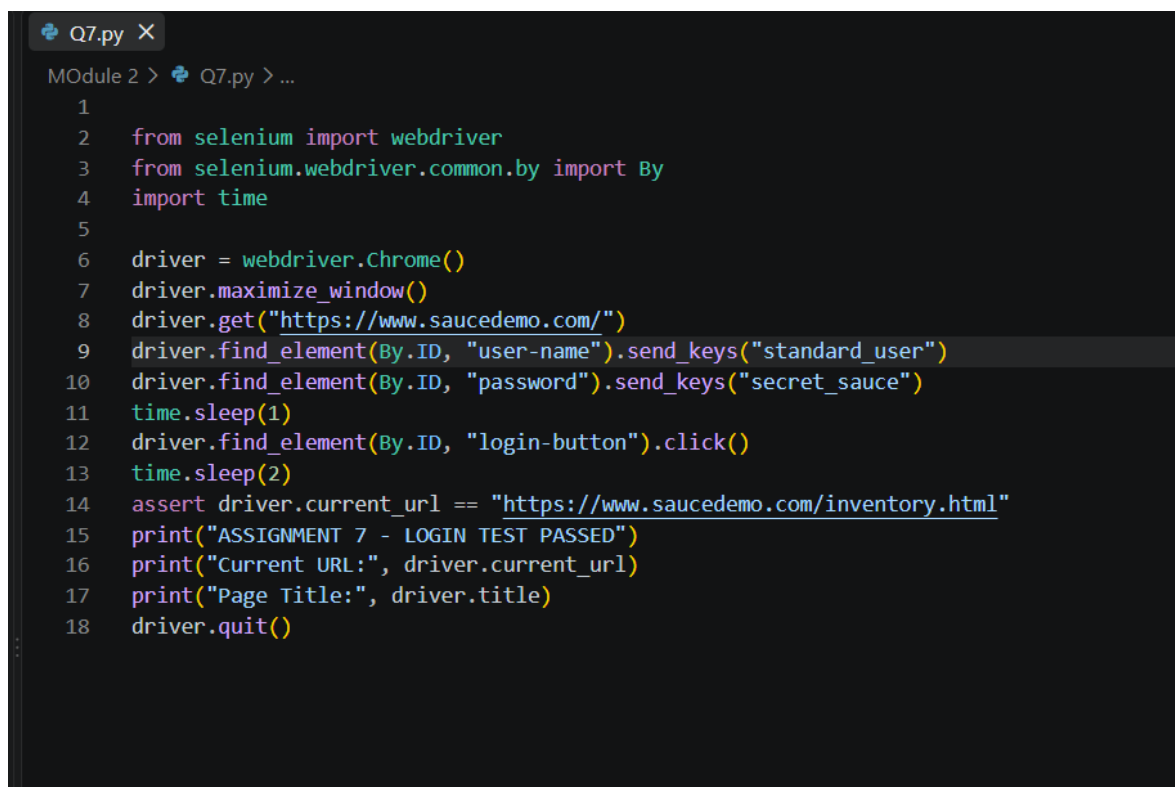
Locate the username, password and login button elements.

Enter the valid credentials and submit the form.

Assert that the current URL matches the expected inventory URL.

Print the resulting URL and page title and close the browser.

5. Source Code

A screenshot of a code editor window titled 'Q7.py'. The code is a Python script using Selenium WebDriver to automate a login process. It imports webdriver and By from selenium, and time. It creates a Chrome driver, maximizes the window, and navigates to 'https://www.saucedemo.com/'. It then finds the 'user-name' and 'password' elements and sends the keys 'standard_user' and 'secret_sauce' respectively. After a 1-second sleep, it clicks the 'login-button'. After another 2-second sleep, it asserts that the current URL is 'https://www.saucedemo.com/inventory.html'. Finally, it prints 'ASSIGNMENT 7 - LOGIN TEST PASSED', the current URL, and the page title, before quitting the driver.

```
1
2 from selenium import webdriver
3 from selenium.webdriver.common.by import By
4 import time
5
6 driver = webdriver.Chrome()
7 driver.maximize_window()
8 driver.get("https://www.saucedemo.com/")
9 driver.find_element(By.ID, "user-name").send_keys("standard_user")
10 driver.find_element(By.ID, "password").send_keys("secret_sauce")
11 time.sleep(1)
12 driver.find_element(By.ID, "login-button").click()
13 time.sleep(2)
14 assert driver.current_url == "https://www.saucedemo.com/inventory.html"
15 print("ASSIGNMENT 7 - LOGIN TEST PASSED")
16 print("Current URL:", driver.current_url)
17 print("Page Title:", driver.title)
18 driver.quit()
```

Fig 7.1 — Python Selenium implementation for login automation.

6. Output

```
Problems Output Debug Console Terminal Ports
• (venv) PS D:\PythonAutomation\Assignments\Module 2> python Q7.py
ASSIGNMENT 7 - LOGIN TEST PASSED
Current URL: https://www.saucedemo.com/inventory.html
Page Title: Swag Labs
❖ (venv) PS D:\PythonAutomation\Assignments\Module 2> |
```

Fig 7.2 — Terminal output confirming successful login and correct redirect URL.

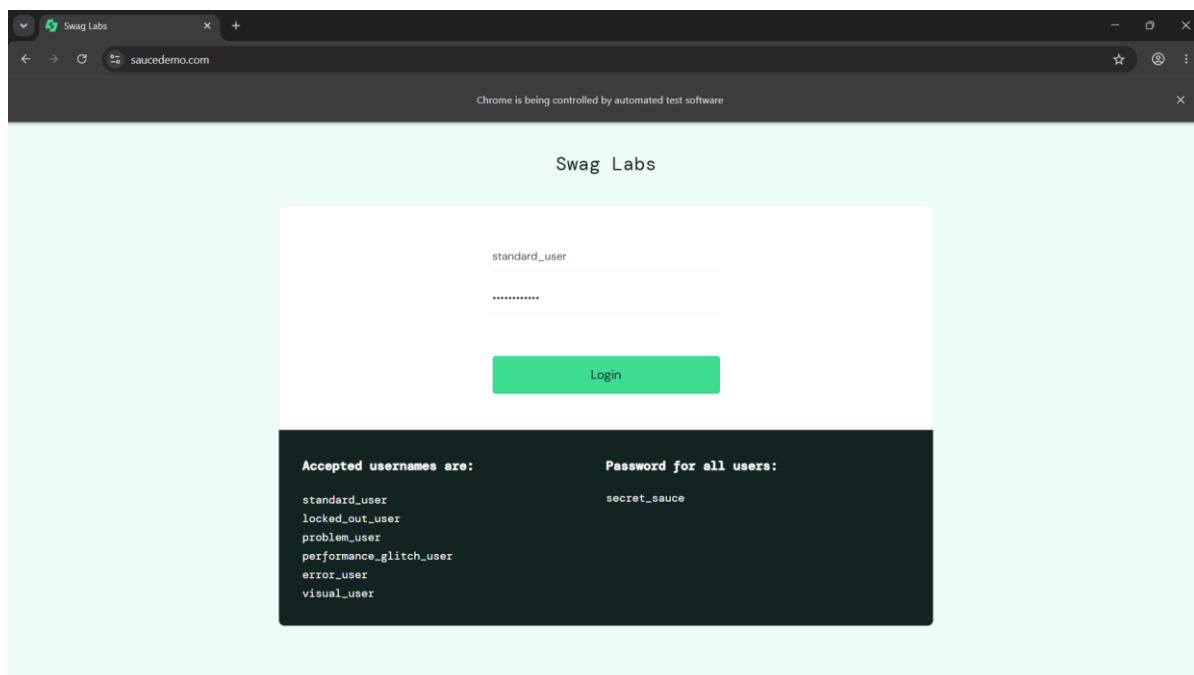


Fig 7.3 — Browser session showing the login form and entered credentials.

7. Result, Observation and Conclusion

The login flow was automated successfully and the post-login inventory URL was validated through an assertion. The terminal output also displayed the resulting URL and page title, confirming that the browser reached the expected application state.

Experiment 8: Data-Driven Testing (DDT) using External CSV

1. Problem Statement

Hardcoding one username and password combination inside a login script limits reuse. The login flow should instead read multiple test cases from an external CSV file, execute the same steps for each row, capture the resulting validation message and compare it with the expected error specified in the data.

2. Objective

- Read multiple login test cases from an external CSV file.
- Execute the same Selenium login flow for each row.
- Capture the validation message displayed by the application.
- Assert that the observed message contains the expected text for that row.

3. Tools, Software and Concepts Used

- Python, Selenium WebDriver and the csv module.
- External CSV test data.
- SauceDemo practice application.
- Loop-based data-driven execution.

4. Implementation Steps

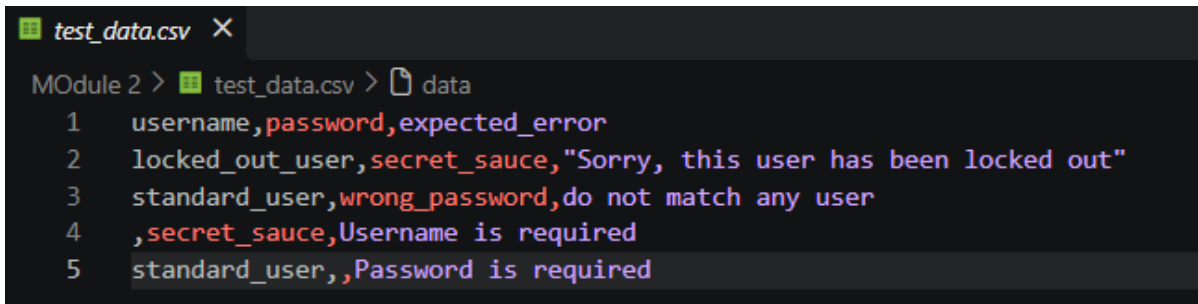
- Create test_data.csv with username, password and expected_error columns.
- Read the CSV using csv.DictReader().
- Loop through each row and open the SauceDemo application.
- Enter the row's username and password and click Login.
- Read the displayed error message and compare it with the expected text.
- Close the browser after the data-driven execution.

5. Source Code

```
Q8.py X
Module 2 > Q8.py > ...
1  import csv
2  import time
3  from selenium import webdriver
4  from selenium.webdriver.common.by import By
5
6  driver = webdriver.Chrome()
7  driver.maximize_window()
8  print("Chrome should be open now")
9
10 with open("test_data.csv") as f:
11     rows = list(csv.DictReader(f))
12
13 for row in rows:
14     driver.get("https://www.saucedemo.com/")
15     driver.find_element(By.ID, "user-name").send_keys(row["username"])
16     driver.find_element(By.ID, "password").send_keys(row["password"])
17     driver.find_element(By.ID, "login-button").click()
18     time.sleep(3)
19     error = driver.find_element(By.CSS_SELECTOR, "h3[data-test='error']").text
20     print(row["username"], "/", row["password"], "->", error)
21     assert row["expected_error"] in error
22
23 print("ASSIGNMENT 8 - DDT LOGIN TEST PASSED")
24 time.sleep(3)
25 driver.quit()
26
```

Fig 8.1 — Python Selenium implementation for CSV-driven login testing.

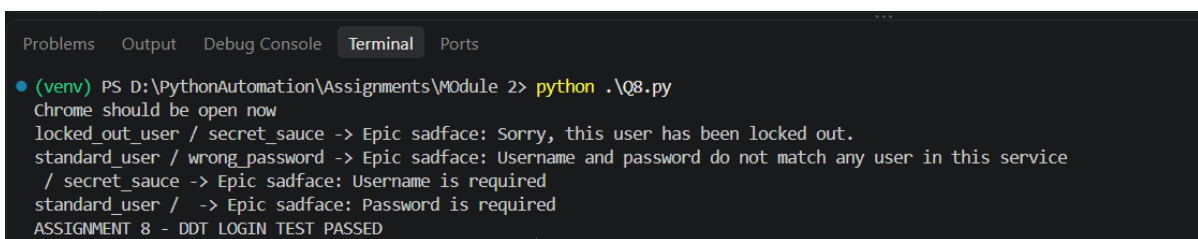
6. Test Data



```
test_data.csv X
MModule 2 > test_data.csv > data
1 username,password,expected_error
2 locked_out_user,secret_sauce,"Sorry, this user has been locked out"
3 standard_user,wrong_password,do not match any user
4 ,secret_sauce,Username is required
5 standard_user,,Password is required
```

Fig 8.2 — External CSV file containing login combinations and expected validation messages.

7. Output



```
Problems Output Debug Console Terminal Ports
• (venv) PS D:\PythonAutomation\Assignments\Module 2> python .\Q8.py
Chrome should be open now
locked_out_user / secret_sauce -> Epic sadface: Sorry, this user has been locked out.
standard_user / wrong_password -> Epic sadface: Username and password do not match any user in this service
/ secret_sauce -> Epic sadface: Username is required
standard_user / -> Epic sadface: Password is required
ASSIGNMENT 8 - DDT LOGIN TEST PASSED
```

Fig 8.3 — Terminal output showing the data-driven login iterations and successful assertions.

8. Browser Evidence

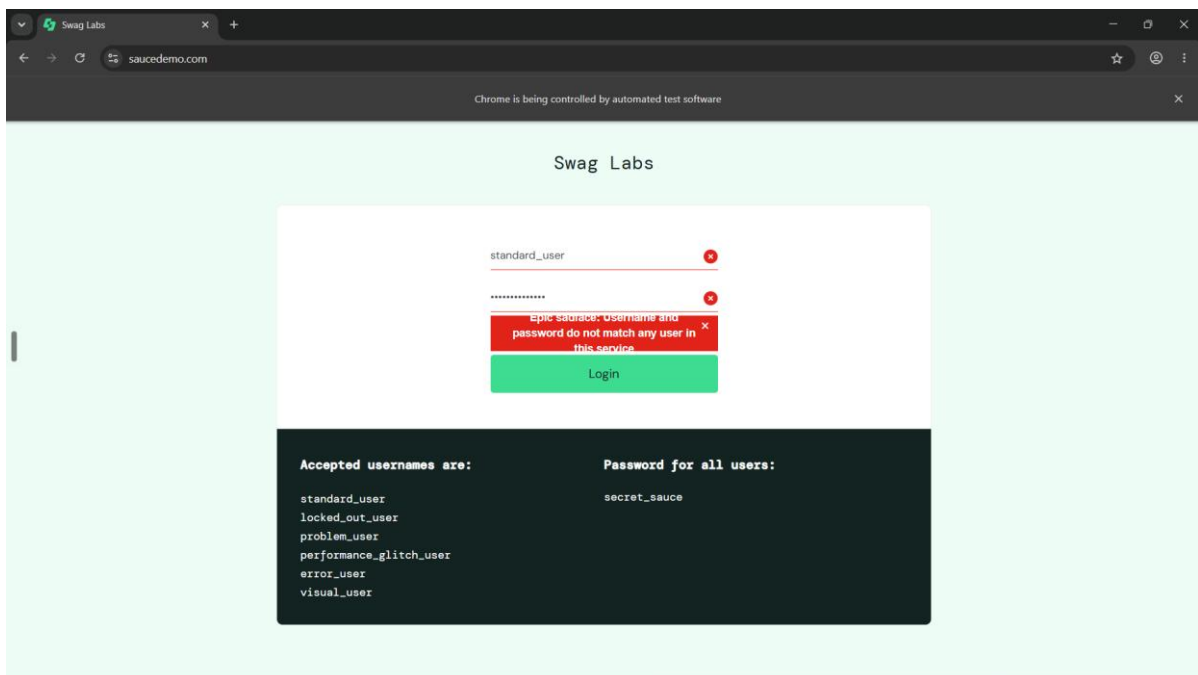


Fig 8.4 — Browser evidence for a data-driven iteration showing the validation error.

8. Result, Observation and Conclusion

The external CSV allowed multiple login scenarios to be executed through the same Selenium flow without duplicating the automation steps. The terminal output shows the three supplied combinations and their corresponding validation messages, and the script completed with the DDT login test marked as passed.

Experiment 9: PyTest Integration with HTML Reporting

1. Problem Statement

The Selenium login checks need to be converted into PyTest test functions so that they can be discovered and executed by the PyTest runner. The execution should also produce a self-contained HTML report summarizing the test outcomes.

2. Objective

Convert the login checks into PyTest test functions.

Execute the tests using the PyTest command-line runner.

Generate a self-contained HTML execution report using pytest-html.

3. Tools, Software and Concepts Used

Python and Selenium WebDriver.

PyTest test runner.

pytest-html plugin.

SauceDemo practice application.

4. Implementation Steps

Create PyTest test functions for the valid and invalid login scenarios.

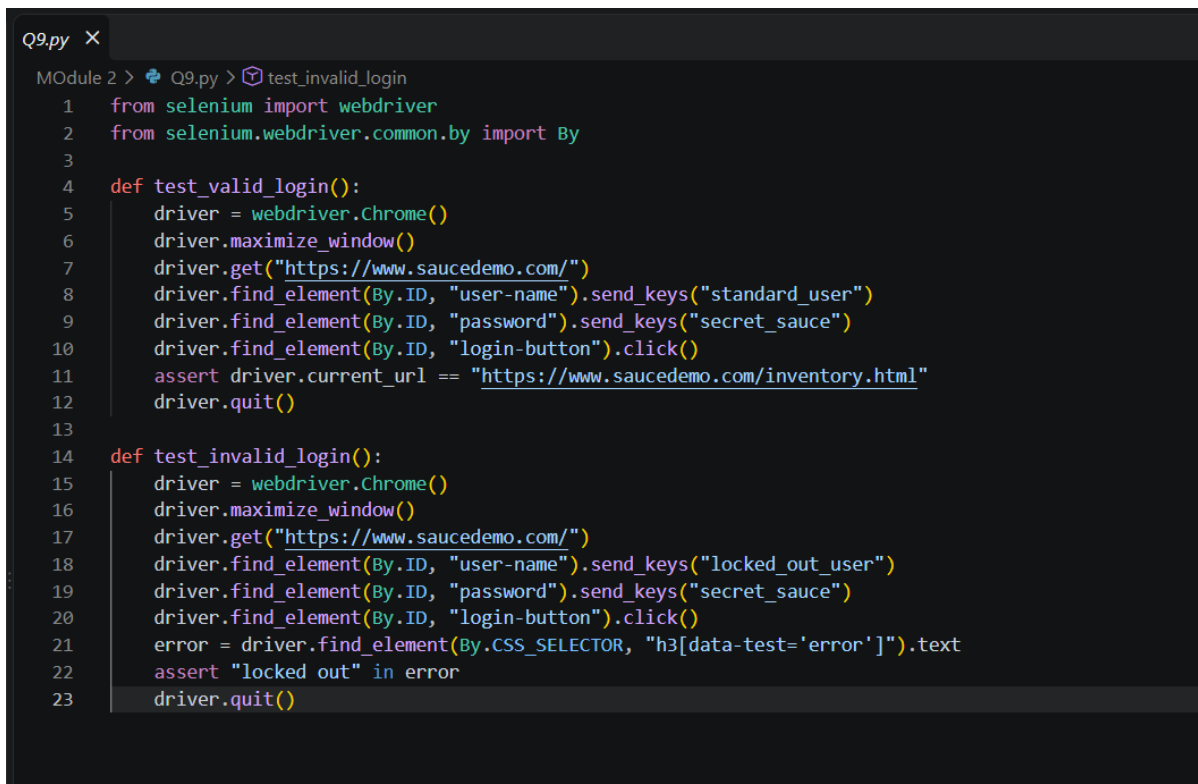
Launch the browser, perform the required login interaction and apply assertions.

Close the browser after each test.

Run the file with PyTest and the pytest-html reporting options.

Review the generated self-contained HTML report.

5. Source Code



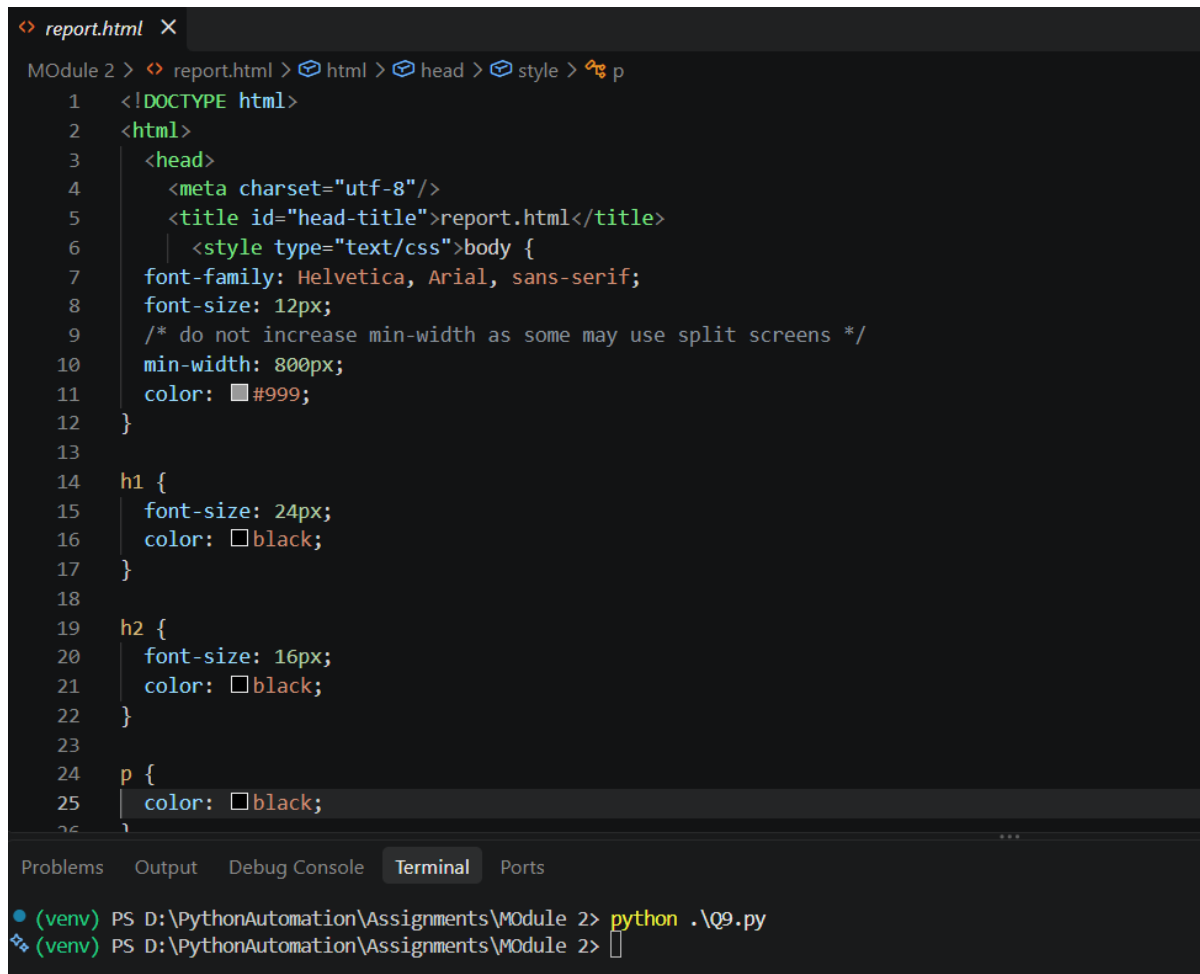
```
Q9.py X
Module 2 > Q9.py > test_invalid_login
1  from selenium import webdriver
2  from selenium.webdriver.common.by import By
3
4  def test_valid_login():
5      driver = webdriver.Chrome()
6      driver.maximize_window()
7      driver.get("https://www.saucedemo.com/")
8      driver.find_element(By.ID, "user-name").send_keys("standard_user")
9      driver.find_element(By.ID, "password").send_keys("secret_sauce")
10     driver.find_element(By.ID, "login-button").click()
11     assert driver.current_url == "https://www.saucedemo.com/inventory.html"
12     driver.quit()
13
14     def test_invalid_login():
15         driver = webdriver.Chrome()
16         driver.maximize_window()
17         driver.get("https://www.saucedemo.com/")
18         driver.find_element(By.ID, "user-name").send_keys("locked_out_user")
19         driver.find_element(By.ID, "password").send_keys("secret_sauce")
20         driver.find_element(By.ID, "login-button").click()
21         error = driver.find_element(By.CSS_SELECTOR, "h3[data-test='error']").text
22         assert "locked out" in error
23         driver.quit()
```

Fig 9.1 — PyTest implementation containing valid and invalid login tests.

6. Run Command

```
pytest -s Q9.py --html=report.html --self-contained-html
```

7. Output



The screenshot displays an IDE window with a file named `report.html`. The editor shows the HTML source code for the report, which includes a DOCTYPE declaration, a head section with a title and a style block, and a body section with CSS rules for h1, h2, and p elements. The style block specifies a font family of Helvetica, Arial, sans-serif, a font size of 12px, a minimum width of 800px, and a color of #999. The h1 rule specifies a font size of 24px and a color of black. The h2 rule specifies a font size of 16px and a color of black. The p rule specifies a color of black. The terminal window at the bottom shows the command `python .\Q9.py` being executed in a virtual environment (venv) at the path `D:\PythonAutomation\Assignments\Module 2`.

```
Module 2 > report.html > html > head > style > p
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <meta charset="utf-8"/>
5   <title id="head-title">report.html</title>
6   <style type="text/css">body {
7     font-family: Helvetica, Arial, sans-serif;
8     font-size: 12px;
9     /* do not increase min-width as some may use split screens */
10    min-width: 800px;
11    color: #999;
12  }
13
14  h1 {
15    font-size: 24px;
16    color: black;
17  }
18
19  h2 {
20    font-size: 16px;
21    color: black;
22  }
23
24  p {
25    color: black;
26  }
```

```
(venv) PS D:\PythonAutomation\Assignments\Module 2> python .\Q9.py
(venv) PS D:\PythonAutomation\Assignments\Module 2>
```

Fig 9.2 — Generated HTML report source and terminal execution evidence.

8. Result, Observation and Conclusion

The login checks were structured as PyTest test functions and the execution command was configured to generate a self-contained HTML report. The resulting report file was produced alongside the test execution, providing a persistent record of the run.

Overall Summary

Milestone 2 covered three practical stages of Selenium/Python automation. Assignment 7 demonstrated a direct login automation flow and URL validation. Assignment 8 extended the same login scenario into data-driven testing using an external CSV file. Assignment 9 introduced PyTest-based test discovery and execution together with self-contained HTML reporting.

Together, the assignments demonstrate the progression from a single Selenium script to reusable test-data execution and a test-runner-based workflow. The practical evidence in this record includes Python implementation screenshots, external CSV data, browser sessions, terminal results and HTML reporting output.

Key Learning Outcomes

- Automating browser interaction and validating application state using Selenium WebDriver.

- Separating test data from automation logic through an external CSV file.

- Executing multiple input combinations through a common test flow.

- Writing Selenium checks as PyTest test functions.

- Generating a self-contained HTML execution report.